Intrusion Detection System Based on In-Depth Understandings of Industrial Control Logic

Motong Sun, Yingxu Lai, *Member, IEEE*, Yipeng Wang, *Member, IEEE*, Jing Liu, *Member, IEEE*, Beifeng Mao, and Haoran Gu

***Abstract*—In industrial control systems (ICSs), intrusion detection is a vital task. Conventional intrusion detection systems (IDSs) rely on manually designed rules. These rules heavily depend on professional experience, thereby making it challenging to represent the increasingly complicated industrial control logic. Although deep learning-based approaches provide better accuracy than other methods, they can only provide alerts. However, they cannot provide administrators with detailed information. In this study, we propose the logic understanding IDS (LU-IDS), which is a rule-based IDS with in-depth understandings of industrial control logic. Our proposed LU-IDS uses a specially designed deep learning-based model to capture features automatically and carry out attack classification. More importantly, it analyzes the knowledge learned from the classification of attacks to understand the abnormal industrial control logic and generate rules. The experimental results indicate that our proposed LU-IDS demonstrates excellent performance on intrusion detection. The rules generated by our proposed LU-IDS can be used to successfully detect all types of attacks on two public datasets.**

***Index Terms*—Control logic, industrial control systems (ICSs), intrusion detection systems (IDSs), logic attribution, rule generation.**

I. Introduction

THE openness and standardization of industrial control systems (ICSs) strengthen their connection with the outside world and profoundly transform conventional industries, such as manufacturing, aviation, and power generation [1]. However, such transformations result in increased security risks facing ICSs. When exploited by an attacker, security vulnerabilities in ICSs may result in huge losses. For example, in February 2020, the cybersecurity and infrastructure security agency revealed that a natural gas pipeline company was attacked using ransomware [2], which uses encryption technology to disable data assets or computing resources and extort the victim for money. The attack resulted in the company's natural gas pipeline equipment being shut down for two days, thereby causing extensive damage to productivity and revenue. Therefore, intrusion detection is vital for ensuring the security of ICSs.

To protect ICSs from attacks, it is essential to establish intrusion detection systems (IDSs). The mainstream technological approach of IDSs involves matching abnormal behavior according to the manually designed rules [3]. Unfortunately, the quality of the rules limits the accuracy of detection. This is because manually designed rules heavily depend on professional experience, thereby making it challenging to represent the increasingly complicated industrial control logic. In contrast, machine and deep learning-based intrusion detection technology provides improved accuracy [4], [5], [6], [7], [8], [9]. However, low computing power and the lack of resources make it almost impossible for ICSs to deploy machine and deep learning-based models directly. Moreover, machine and deep learning-based models can only provide alerts, but they cannot provide administrators with detailed information, such as the physical devices in the factory that are being attacked or those that are malfunctioning. Therefore, such models cannot be used to guide administrators to fix anomalies promptly.

In this study, we propose the logic understanding IDS (LU-IDS), which is an IDS based on rule matching. Our proposed LU-IDS can automatically construct intrusion detection rules by deeply understanding the knowledge learned by a deep learning-based model. Our proposed system is based on the key insight that the quality of manually designed rules limits accuracy, and machine or deep learning-based models are difficult to deploy in ICSs directly. Therefore, we design a deep learning-based model for automatically capturing the logical relationships between all sensors and actuators in ICSs. As a result, we can extract critical control logic from the logical relationships to construct intrusion detection rules. In addition to detecting attacks, such rules contain more detailed information than manually designed rules, thereby helping administrators to promptly fix anomalies resulting from attacks.

Manuscript received 13 July 2021; revised 14 December 2021 and 15 February 2022; accepted 8 August 2022. Date of publication 22 August 2022; date of current version 3 March 2023. This work was supported in part by the National Key R&D Program of China (Key Technologies and Applications of Security and Trusted Industrial Control System) under Grant 2020YFB2009500, and in part by the Natural Science Foundation of Beijing Municipality under Grant 19L2020. Paper no. TII-21-2959. *(Corresponding author: Yingxu Lai.)*

Motong Sun is with the Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China (e-mail: sunmt19@emails.bjut.edu.cn).

Yingxu Lai is with the Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China, and also with the Engineering Research Center of Intelligent Perception and Autonomous Control, Ministry of Education, Beijing 100124, China (e-mail: laiyingxu@bjut.edu.cn).

Yipeng Wang, Jing Liu, Beifeng Mao, and Haoran Gu are with the Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China (e-mail: yipeng.wang1@gmail.com; jingliu@bjut.edu.cn; maobf19@emails.bjut.edu.cn; ghrvip@emails.bjut.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2022.3200363.

Digital Object Identifier 10.1109/TII.2022.3200363

We design three modules for our proposed LU-IDS. 1) The tensor construction module: It can build two-dimensional (2-D) measured element tensors by embedding all the sensor and actuator values in each moment. 2) The logic learning module: It can automatically learn the logical relationship between the sensors and actuators by training a deep learning-based model that is competent in carrying out attack classification tasks. 3) The rule generation module: It can analyze the knowledge learned by a deep learning-based model and generate interpretable rules with high confidence to detect attacks.

The main contributions of this study are as follows:

1) We propose a novel IDS based on the in-depth understandings of deep learning-based models. Our proposed IDS takes all the sensor and actuator values as input, and it outputs rules for intrusion detection. Moreover, these interpretable rules can help administrators cope with various types of attacks.
2) We use embedding vectors to represent the sensor and actuator values, and we design a specific deep learning-based model for capturing the logical relationship between these values automatically. The most prominent contribution is that we can obtain effective and interpretable rules by analyzing the knowledge learned by the model during the attack classification process.
3) The excellent performance of our logic learning module on attack classification indicates that our proposed LU-IDS can capture the complex industrial control logic. Moreover, the experimental results indicate that our generated rules can be used to successfully detect all types of attacks on the two public datasets with a low FAR.

The remainder of this article is organized as follows: The background is discussed in Section II. The related works are discussed in Section III. The specific approach is described in Section IV. The experiments and the results are presented and discussed in Section V. Finally, Section VI concludes this article.

II. Background

ICSs mainly comprise control centers and field sites. The field site contains sensors, actuators, and programmable logic controllers (PLCs), which are used to monitor and control industrial procedures [10]. The structure of a PLC is simple. It includes the input and output modules, the power supply, and the memory for storing control logic programs. The input module perceives the state of an ICS by receiving sensor values. The PLC then sends signals to the actuator according to the control logic programs to control the physical process.

Nowadays, faced with numerous security risks, most PLCs and hardware devices used in ICSs do not have self-protection mechanisms. Therefore, it is essential to deploy IDSs in ICSs. However, it is challenging to protect ICSs at the hardware or PLC level because their structure is simple. Moreover, each PLC can only monitor the hardware device under its control, thereby limiting the detection performance of the IDS deployed on the PLC. Therefore, IDSs are usually deployed on the bypasses of industrial control networks.

Attacks against ICSs are generally achieved through the injection of malicious code and through tampering with sensor or actuator values. When an attack occurs, the abnormal control logic manifests as the incongruity between the system states, i.e., the sensor values, and the control signals, i.e., the actuator values. For example, if the gas injection operation is performed when the gas pressure is close to the upper threshold of the reaction pipeline, the pipeline will burst. During such an attack, the abnormal control logic is reflected in the incongruity between the gas pressure and the injection operation. However, the industrial control logic becomes increasingly complicated, thereby making it difficult for experts to design effective rules for detecting attacks. Therefore, in this study, we propose a novel IDS based on the rules constructed by analyzing the knowledge learned by the deep learning-based models.

III. Related Works

Currently, the most commonly applied technique for detecting intrusions in ICSs is rule matching, which benefits from the flexibility of deploying rules and the low resource requirements for detecting attacks. This technique relies on manually constructed rules to monitor the performance of ICSs in real-time. Once the traffic or physical state of the ICS matches the content of the rules, this technique will alert the administrators. Obviously, the rule-based technique relies heavily on professional knowledge and the quality of the rules.

Adepu et al. [3] proposed a distributed attack detection method, which could detect attacks by identifying abnormal physical behaviors or operations in factories. The proposed method identified anomalies by constructing a set of rules based on invariants in the production procedure employed by the factory. Although such manually designed rules are versatile, they make it challenging to detect attacks involving highly complicated industrial processes or control logic. For example, Adepu and Mathur [3] admitted that it was difficult for them to determine the invariants of industrial chemical reactions, such as reagent concentration, and to design rules based on such reactions.

Owing to advancements in machine learning, an increasing number of researchers have committed themselves to combining machine learning-based approaches with industrial intrusion detection methods [11], [12], [13]. Lin et al. [14] proposed a one-class graphical classifier based on the normal behavior of ICSs. The proposed classifier can be used to locate abnormal sensors and actuators based on the interpretability of the graphical models. However, the inevitable connection between sensors and actuators in ICSs makes it challenging to satisfy the variable independence assumption in Naive Bayes classifiers. Inoue et al. [15] constructed an intrusion detection model based on a one-class support vector machine (OCSVM) according to the legitimate physical states of ICSs. Compared to the deep neural networks (DNNs) proposed in [15], the OCSVM can be used to detect an additional number of anomalies, and it requires a lower computation cost.

Over the recent years, deep learning-based intrusion detection methods have become popular. Shalyga et al., in the work of [16], designed an autoencoder (AE) model based on the multilayer perceptron (MLP) to detect attacks on ICSs. Kravchik et al. [17] proposed an intrusion detection method, which is based on the

deviation between the predicted values and the observed values of measured industrial elements. The proposed method could be applied to different convolutional neural networks (CNNs) and recurrent neural networks (RNNs). Abdelaty et al. [18] proposed an intrusion detection framework based on wide and DNNs and greedy algorithms. Abdelaty et al. [4] improved the framework mentioned above by replacing the output sections with two different neural networks. The proposed framework can automatically update its parameters without human intervention.

In [19], a 1-D CNN was used to detect anomalies in ICSs, and the experimental results were compared to those of the principal component analysis (PCA) and two AE models that had been proposed previously. Li et al. [20] proposed an unsupervised industrial intrusion detection framework based on generative adversarial networks (GANs) in which the generator and the discriminator comprise long short-term memory (LSTM) networks for capturing the temporal relationships between ICS data. Although the proposed framework can learn the potential interactions between different variables in ICSs, the length of the sequence that is input into the LSTM affects its stability. Farsi et al. [21] proposed an online anomaly detection method based on the Euclidean distance between the current and the latest states of an ICS. The proposed method can effectively reduce false positives, which benefit from its adaptability to the fluctuation of ICS states within the normal range.

In summary, the two mainstream industrial intrusion detection techniques have their advantages and disadvantages. Although techniques based on rule matching are easy to deploy and quick to detect, the quality of manually designed rules always limits their detection accuracy. Compared with the rule-based technique for intrusion detection in ICSs, the machine or deep learning-based methods demonstrate enhanced detection accuracy and lower FARs. However, low computing power and the lack of resources make it almost impossible for ICSs to deploy these machine or deep learning-based models directly. Therefore, an unavoidable limitation of the machine or deep learning-based methods is deploying them in ICSs.

To address the limitation mentioned above, we propose the LU-IDS. Our proposed LU-IDS can construct intrusion detection rules by analyzing the knowledge learned by deep learning-based models. Therefore, the proposed LU-IDS can consider the advantages of both rule-based and model-based intrusion detection methods. Although a few machine and deep learning-based methods can be used to locate abnormal sensors or actuators in ICSs, the detection results generated by most of these methods are not interpretable. Therefore, such models cannot be used to guide administrators to fix anomalies promptly. Contrarily, our generated rules are interpretable and contain more detailed information than manually designed rules.

IV. Method of Our Proposed LU-IDS

In this section, we begin with an overview of our proposed LU-IDS, after which we introduce the basic structure and technical principles of the detailed modules in the LU-IDS.

Algorithm 1: The Algorithm of Our Proposed LU-IDS.

Input: The set of sensor values S and actuator values A.
Output: The intrusion detection rules with confidence.

for each $s \in S, a \in A$ **do**
  Input s, a into tensor construction module to get the measured element tensor set T.
end for
Train the deep network structure M in the logic learning module with T.
for each $t \in T$ **do**
  Input t into M again to obtain the feature maps f.
  Input f into the rule generation module to construct the intrusion detection rule r.
end for
Construct the rule sets R with r.
Detect the attacks against ICS with R.
return detection results.

A. Overview of the LU-IDS

The LU-IDS is a hierarchical IDS, and it comprises three modules, as shown in Fig. 1. Compared to other industrial IDSs, our proposed LU-IDS has the following two advantages: First, it relies on a specially designed deep learning-based model for automatically capturing the relationships between sensors and actuators in ICSs. Therefore, it can effectively classify different types of attacks. Second, it can generate intrusion detection rules without the need for human intervention. The rules are constructed by deeply analyzing the knowledge, i.e., the abnormal industrial control logic, learned by the deep learning-based model. The rules that contain highly relevant logic can also help administrators in preventing various types of attacks. Algorithm 1 shows the workflow of our proposed LU-IDS. In the next part of this section, we shall describe each module of our proposed system in detail.

B. Tensor Construction Module Based on the Embedded Expression of ICS Measured Elements

As described in Section II, attacks against ICSs generally result in the logical incongruity between the system states and the control signals, i.e., the logical incongruity between the sensor and actuator values. Therefore, we can train a deep learning-based model to learn different logical relationships between sensors and actuators corresponding to the normal and abnormal states of ICSs. Fig. 2 shows the structure of the tensor construction module based on the embedded expression of ICS measured elements.

Given an ICS containing n subsystems $I = \{b_1, b_2, ..., b_n\}$, at time t, the sensor and actuator values contained in subsystem b_j are $S(t) = \{s_1, s_2, ..., s_p\}$ and $A(t) = \{a_1, a_2, ..., a_q\}$, where p and q represent the number of sensors and actuators, respectively. First, we encode the floating-point value s_i of each sensor into a hexadecimal bytes tensor $\{s_{i1}, s_{i2}, s_{i3}, s_{i4}\}$ according to the IEEE 754 standard. We then concatenate all the encoded sensor values $S(t)'$ and actuator values $A(t)$ in each subsystem b to

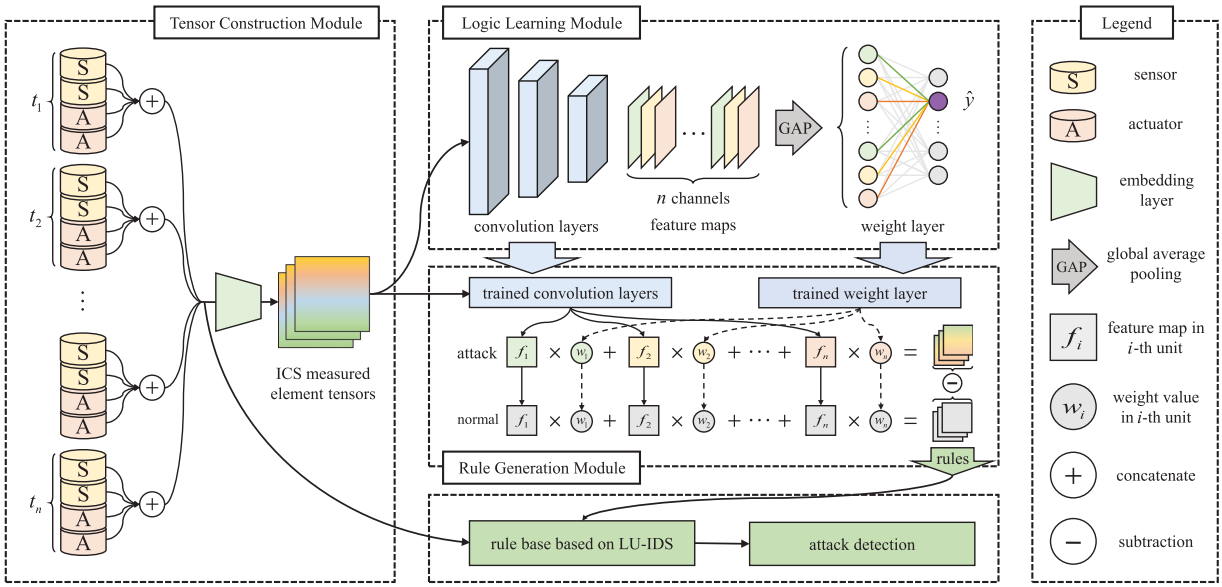


Fig. 1. Overall structure of our proposed LU-IDS.

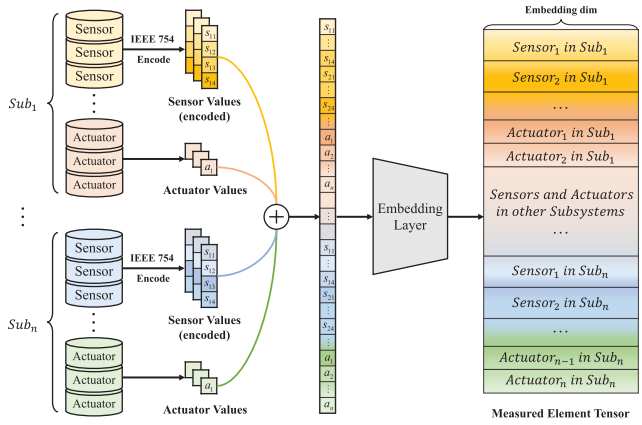


Fig. 2. Structure of the tensor construction module.

obtain a 1-D vector z. Assuming that the length of z is l and the dimension of the embedding vector is d, the embedding layer can be regarded as a matrix $E \in \mathbb{R}^{l \times d}$. Note that the matrix E can be updated continuously during the training process. Given a specific element c in z, the embedding vector of c is the $c-$ th row in E. After entering vector z into the embedding layer, we can finally obtain the 2-D measured element tensor f at time t.

Compared to directly concatenating the sensor and actuator values into a 1-D tensor to train a deep learning-based model, the 2-D measured element tensors have the following three advantages: 1) Most of the sensor values in ICSs undergo very few changes over a short time. Encoding them can effectively highlight this type of change in the data. 2) The embedding vectors enrich the information expressed by the sensor and actuator values, and they avoid conflicts in which similar values in a sequence may express different meanings. 3) Our proposed model can learn specific representations of the embedding vectors of different sensor and actuator values through the trainable embedding layer, thereby improving classification performance.

C. Logic Learning Module Based on the Deep Learning-Based Model Used for Attack Classification

Fig. 3 shows the structure of the logic learning module based on the deep learning-based model. This module takes 2-D ICS measured element tensors as input, and it uses convolution layers with a kernel size of $n \times 1$ to extract features from them automatically. Compared to the kernels of size $n \times n$ in the conventional convolution layers, the kernels of size $n \times 1$ are more suitable for extracting the information required from the measured element tensors. This is because the vertical dimensions of these tensors comprise sensors and actuators at a specific moment. In contrast, the horizontal dimension is the embedding vector of these measured elements, thereby making it meaningless for the use of convolution to extract features in this dimension. Therefore, we choose kernels with a size of $n \times 1$ to capture the logical relationship between sensors and actuators.

We hope to design a module that contains a deeper network structure to extract high-dimensional features from the ICS measured element tensors. This is because such features can help in characterizing the logical relationship between the sensors and actuators more effectively. Inspired by [22], we add residual connections to solve the issue of gradient disappearance resulting from network deepening. As shown in Fig. 3, the convolution layer with a kernel size of 1×1 is used to increase the dimensionality of the feature maps and down-sample their size. In addition, we design a weight layer, which is connected to the convolution layer through the global average pooling layer, to complete the classification process. In the following paragraphs, we describe the specific process of attack classification in detail.

Given a measured element tensor f at a specific time when attack c occurs, we use $f_k(x, y)$ to describe the activation of

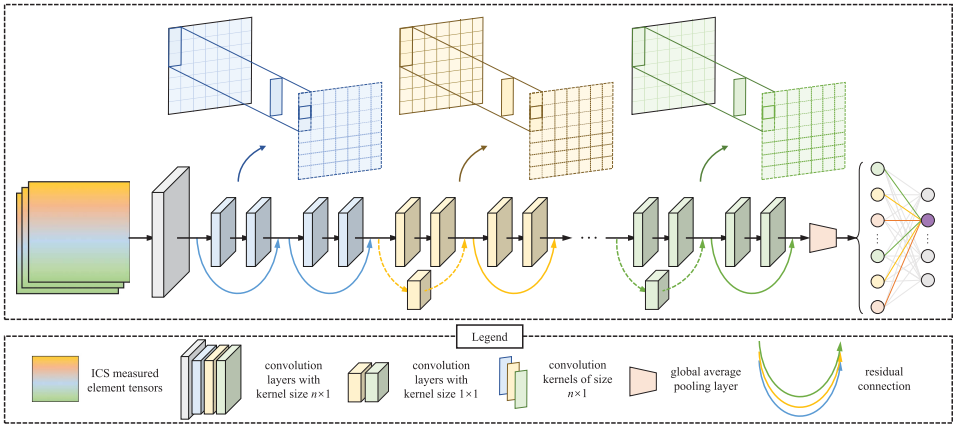


Fig. 3. Structure of the logic learning module.

dimension k in the last convolution layer at spatial position (x, y). The global average pooling layer is used to down-sample the dimensionality of the feature maps. Next, we calculate the degree to which tensor f conforms to the attack category c using the following formula:

$$S_{cf} = \sum_{k} w_k^c \sum_{x,y} f_k(x, y) \tag{1}$$

where w_k^c represents the weight of (k, c) in the weight matrix and describes the importance of dimension k when the tensor is classified into category c. We then calculate the probability of classifying f into category c as follows:

$$P_{cf} = \frac{\exp(S_{cf})}{\sum_c \exp(S_{cf})}. \tag{2}$$

Finally, we choose the attack category with the highest probability as the final classification result of f.

D. Rule Generation Module Based on the In-Depth Understanding of Industrial Control Logic

Rather than designing intrusion detection rules based on the experience of domain experts, we hope to generate highly effective and targeted rules according to the abnormal control logic resulting from different types of attacks. The logic learning module can automatically learn and extract the abnormal control logic corresponding to different attacks because it realizes the highly efficient attack classification. Therefore, to understand this type of abnormal control logic, we construct attack-specific mappings for analyzing the decision basis of the logic learning module. Fig. 4 shows the structure of the rule generation module.

1) Constructing the class activation mappings for attack and normal samples: Given a measured element tensor f corresponding to an attack or a normal sample, we input it into the trained convolution layers again and extract feature maps of different dimensions in the last convolution layer. We then perform a weighted linear sum on them according to the weight values in the weight layer. The class activation mapping of f can be calculated as follows:

$$m_{cf}(x, y) = \sum_{k} w_k^c f_k(x, y). \tag{3}$$

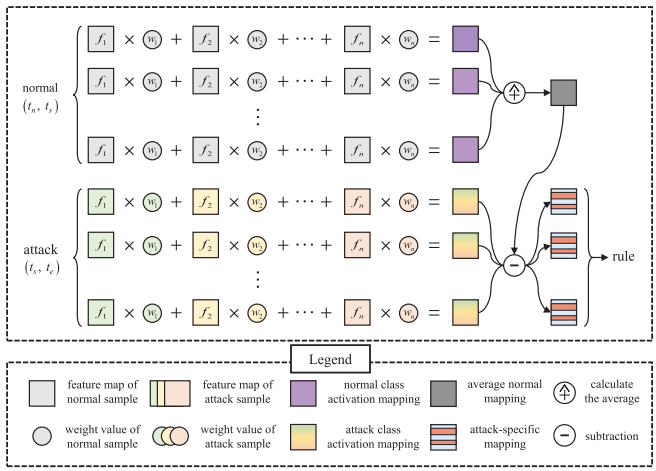


Fig. 4. Structure of the rule generation module.

The formula presented above is obtained by performing identity transformation on (1). Therefore, this mapping can directly represent the importance of activation at (x, y) when f is classified into c. Intuitively, we cannot ensure that the feature map in each dimension is activated by a specific embedding vector of one sensor or one actuator value. Therefore, instead of simply using layer-wise relevance propagation [23] to determine the dimension that contributes the most, we perform a weighted linear sum of all activations, i.e., feature maps, to analyze the decision basis in attack classification. By upsampling the class activation mappings to the size of the original measured element tensors through bilinear interpolation, we can distinguish the most relevant sensors and actuators for each type of attack.

2) Constructing the attack-specific mappings: The time sequence of the industrial control logic is rigorous. Therefore, we further construct the attack-specific mappings to highlight the difference in industrial control logic between the attacked state and the normal state of the ICS. Given an attack that occurs from time t_s to time t_e, we first calculate the attack class activation mapping m_{at} of the measured element tensor at each time $t \in (t_s, t_e)$. Next, we calculate the average normal

class activation mapping $\overline{M}_{\text{normal}}$, which represents the normal control logic of the ICS during time (t_n, t_s) before the attack occurs, as follows:

$$\overline{M}_{\text{normal}} = \frac{1}{t_s - t_n} \sum_{t_n}^{t_s} m_{nt^*}(x, y) \tag{4}$$

where $m_{nt^*}(x, y)$ represents the normal class activation mapping at time $t^* \in (t_n, t_s)$. Finally, we calculate the attack-specific mapping M_{at} at time t according to the following formula:

$$M_{at} = m_{at} - \overline{M}_{\text{normal}}. \tag{5}$$

3) Generating the detection rules based on the attack-specific mappings: Assuming that (t_s, t_e) contains n time points, we calculate the attack-specific mapping at each time $t \in (t_s, t_e)$ and generate specific detection rules based on the measured elements highlighted in each mapping. First, we set a threshold th to filter out spatial locations with low activation in attack-specific mappings. Next, we overlap each filtered mapping with its corresponding original tensor and calculate the overlap area between the retained high activation part and the respective location of each measured element. Afterward, for each attack-specific mapping M_{at}, we select the first q measured elements with the biggest overlap area to build the attack activation set $\{e_1, e_2, ..., e_q\}$. Therefore, we can obtain n sets according to all the attack-specific mappings from t_s to t_e. Finally, we calculate the top p measured elements that appear most frequently in these sets and generate a specific rule R based on them. R can be described as follows:

$$R = \{e_1 : con_1 \wedge e_2 : con_2 \wedge ... \wedge e_p : con_p\} \tag{6}$$

where $\{con_1, con_2, ..., con_p\}$ represent the judgment conditions corresponding to different measured elements. For the sensors in R, we determine the conditions based on the range of their values (e.g., the value is always higher or lower than a specific threshold) from t_s to t_e. For the actuators in R, we determine the conditions according to their control state, e.g., valve open or closed. Moreover, we define rule confidence as a metric for evaluating the reliability of the rules. The calculation of the confidence r is expressed as follows:

$$r = \frac{1}{p \times n} \sum_{i=1}^{p} n(e_i) \tag{7}$$

where $n(e_i)$ represents the total times the measured element e_i appears in all the attack activation sets. Fundamentally, the rule confidence describes the stability of the rules generated by our proposed LU-IDS. Algorithm 2 shows the workflow of the rule generation module.

Algorithm 2: The Algorithm in Rule Generation Module.

Input: The set of measured element tensors at the time (t_s, t_e) when the attack occurred F_a, the set of measured element tensors at the time (t_n, t_s) before the attack occurred F_n, the attack classification model Ml.

Output: The intrusion detection rules R with confidence r.

for each $f_a \in F_a$, $f_n \in F_n$ **do**
  Input f_a, f_n into Ml to get feature maps fm_a, fm_n.
  Compute m_a, m_n of f_a, f_n according to (3).
end for
Calculate $\overline{M}_{normal}$ according to (4).
for each $f_a \in F_a$ **do**
  Calculate m_{at} at each time $t \in (t_s, t_e)$.
  Calculate M_{at} at time t according to (5).
  Use th to filter out spatial locations with low activation in M_{at} to get filtered mapping MF_{at}.
  Overlap MF_{at} with M_{at} and calculate overlap area.
  Select the first q measured elements with the biggest overlap area to build $S = \{e_1, e_2, ..., e_q\}$.
end for
Compute the top p measured elements that appear most frequently in S generate a rule R.
Compute the rule confidence r according to (7).
return R with r.

V. Experiments and Evaluation

In this section, we evaluate the performance of our proposed LU-IDS in several aspects, including attack classification, rule generation, and intrusion detection. In addition, we describe the datasets, experiment settings, evaluation metrics, experiment results, and we discuss the limitation of the rules generated by our proposed LU-IDS in detail.

A. Datasets

We use two datasets that are used widely in the field of intrusion detection to evaluate the system proposed in this study. The detailed description of each dataset is as follows:

1) SWaT [24]: The SWaT dataset is collected in a six-stage safe water treatment testbed, which represents a scaled-down version of a real-world industrial water treatment plant. This dataset contains seven days of data under normal conditions and four days during which such attacks were conducted. We use 35 attacks with physical impact and put different labels on them according to the specific time when these attacks occurred.

2) WADI [25]: The WADI dataset is collected in a down-scaled water distribution testbed designed by the same researchers who designed the SWaT dataset. This dataset contains data generated by the testbed running continuously for 16 days, whereby the first 14 days involve regular operations and 15 attacks in the following two days. The ultimate goal of these attacks was to prevent water supply to consumer tanks.

An attack point in the SWaT dataset could be a physical element or an entry point through the communication network connecting sensors or actuators to the PLCs and the supervisory control and data acquisition (SCADA) systems. The SWaT dataset comprises six stages, where each stage contains a different number of sensors and actuators. Based on attack points in each stage, the attacks are divided into following four types.

1) Single Stage Single Point (SSSP) refers to the attack that focuses on exactly one point in a cyber physical system (CPS).

2) Single Stage Multi Point (SSMP) refers to the attack that focuses on two or more attack points in a CPS, but only one industrial stage.
3) Multi Stage Single Point (MSSP) refers to the attack that is similar to an SSSP attack, but it is carried out in multiple stages.
4) Multi Stage Multi Point (MSMP) refers to the attack that is similar to an SSMP attack carried out in two or more stages.

B. Experiment Settings

1) Baselines for Attack Classification: We compare the performance of our proposed logic learning module to that of other state-of-the-art (SOTA) methods on the two datasets mentioned above. Considering the fairness of the experiments, we have tried to construct an experimental scene similar to those where LU-IDS is implemented to apply the above SOTA methods. We reproduce the SOTA methods according to the description in their corresponding papers, but we adjust some details so that these methods can better fit the experimental scene. The specific implementation of the above SOTA methods is as follows.

1) W-PCA [19]: We implement the W-PCA using the standard PCA algorithm from the Python scikit-learn package. In addition, we reproduce the overlapping window algorithm in this article. The window size is set to 4, and the sliding step of the window is set to 1. In data preprocessing, we subsample the training and test data of SWaT and WADI datasets at a rate of 5 s and 10 s, respectively. Moreover, we extract the time-domain data of sensors and actuators with normal distributions using the Kolmogorov–Smirnov test. The input of the W-PCA includes 11 sensors and 25 actuators. We calculate the mean absolute error (MAE) of the data during the detection process in each time window and judge whether it exceeds the detection threshold we set. Setting the detection threshold to 2.8 can achieve the best performance.
2) UAE [19]: We construct the model as an AE structure based on linear layers. Both the encoder and decoder in the model contain only one linear layer, and the best results can be achieved using a compact representation that is 50% the size of the input. In addition, the model contains two linear layers to inflate the input and encoded dimension. In data preprocessing, we subsample the datasets and extract the sensors and actuators as the features in the same way as W-PCA. Different from W-PCA, the features are normalized to (0,1) scale. During the training phase, we set the learning rate to 0.0005, the number of epochs to 50, and the batch size to 128. The sequence length of the input data is set to 1. In the detection phase, the detection threshold is set to 2.75. We calculate the mean square error (MSE) between the reconstructed and original data to detect the attack.
3) 1D-CNN [19]: We build the model as a structure with five convolutional layers and three linear layers. The convolutional layers in the model are 1-D convolution operations, and the size of the kernel decreases from 5 to 1, with a stride of 1. Each convolutional layer in the model is connected to the batch normalization layer, the ReLU function, and the max-pooling layer. Additionally, we doubled the filter size with each layer. In data preprocessing, we process both datasets in the same way as UAE. During the training phase, we set the learning rate to 0.001, the number of epochs to 50, and the batch size to 64. The sequence length of the input data is set to 50. In the detection phase, we set the detection threshold to 5.3 and calculate the MSE between the predicted and real data to detect the attack.
4) AADS [18]: We construct the deep module of the model as a structure consisting of two 1-D convolutional layers with kernels of size 2 and stride 1. The wide module of the model is a linear layer. The input size of the model is 60×51, i.e., the input sequence length is 60, and the number of features in each sequence is 51 in SWaT. The output size of the model is 4×25, i.e., the output sequence length is 4, and the number of features is 25 sensor values. In the data processing stage, we divide the training data into windows with a sequence length of 60, and the model will predict the data with a length of four after each window. Additionally, the data are normalized to (0,1) scale. During the training phase, we set the learning rate to 0.001, the number of epochs to 50, and the batch size to 128. Adam is used as a gradient descent optimizer. In the detection phase, we set dynamic thresholds according to the description in this article and calculate the MSE between the predicted and real data to detect the attack.
5) DAICS [4]: DAICS is improved from AADS. Its main improvement is to build a deep learning model to train dynamic thresholds in AADS. We reproduce the dynamic threshold model in DAICS. The model is a CNN with two layers. The convolutional layers in the model are 1-D convolution operations, and the size of the kernel is 2, with a stride of 1. A max-pooling layer is connected after each convolutional layer. In the data processing stage, we process both datasets in the same way as AADS. During the training phase, we set the learning rate to 0.0005, the number of epochs to 100, and the batch size to 32. We use the trained threshold model to generate thresholds to detect attacks in the detection phase.

2) Settings of Our Proposed LU-IDS: In the logic learning module, we design a specific network structure for automatically extracting the logical relationship between sensors and actuators and carry out attack classification. Table I describes the structure of this module in detail. We define K as convolution kernel size, S as stride, C as the number of channels (i.e., filters), P as padding size, N as the number of residual blocks, BN as Batch Normalization, GAP as average pooling layer, and WN as weight layer.

In the rule generation module, we set the threshold to 500/1000/2000 to filter out spatial locations with low activation in the attack-specific mappings. Our LU-IDS construct the rules according to the sensors and actuators corresponding to the top 20/25/30 rows in the measured element tensors with the highest activation. The final rule set is composed of optimal

TABLE I
SPECIAL NETWORK STRUCTURE OF THE LOGIC LEARNING MODULE IN OUR PROPOSED LU-IDS

Layer Names	Specific Structures
Conv	K=(3, 1), S=1, C=32, P=1, BN, ReLU
ResidualBlock1	N=2, [K=(3, 1), S=1, C=32, P=1, BN, ReLU], [K=(3, 1), S=1, C=32, P=1, BN, ReLU]
ResidualBlock2	N=2, [K=(3, 1), S=2, C=128, P=1, BN, ReLU], [K=(3, 1), S=1, C=128, P=1, BN, ReLU]
ResidualBlock3	N=2, [K=(3, 1), S=2, C=256, P=1, BN, ReLU], [K=(3, 1), S=1, C=256, P=1, BN, ReLU]
ResidualBlock4	N=2, [K=(3, 1), S=2, C=512, P=1, BN, ReLU], [K=(3, 1), S=1, C=512, P=1, BN, ReLU]
GAP+WN	weight-size=(512, number of classes)

rules corresponding to different threshold combinations. We set the learning rate to 0.01 and set the batch size to 32. We select the CrossEntropyLoss function and use the Adam algorithm as the optimizer.

C. Evaluation Metrics

We use several metrics to evaluate the performance of our proposed LU-IDS comprehensively. On the one hand, we use accuracy, precision, recall, and the F1-score to evaluate the performance of our logic learning module. These metrics have been used widely for classification tasks, and the details of the metrics are as follows:

$$\begin{aligned} \text{Accuracy} &= \frac{TP+TN}{TP+FP+TN+FN}, \\ \text{Precision} &= \frac{TP}{TP+FP}, \\ \text{Recall} &= \frac{TP}{TP+FN}, \end{aligned} \tag{8}$$

$$F1-\text{score} = \frac{2 \times \text{Precision} \times Recall}{\text{Precision} + \text{Recall}}, \tag{9}$$

where TP, TN, FP, and FN in the equations presented above represent the true positive, the true negative, the false positive, and the false negative, respectively. In the field of intrusion detection, TP indicates a normal sample classified as a normal sample, and TN indicates an attack sample classified as an attack sample. FP indicates an attack sample classified as a normal sample, and FN indicates a normal sample classified as an attack sample.

On the other hand, we use rule confidence to evaluate the reliability of the rules generated by our proposed LU-IDS. The definition and calculation of rule confidence have been described in Section IV-D. Additionally, we calculate the false alarm rates (FAR) for evaluating the performance of intrusion detection with the rules on the two public datasets. The FAR can be calculated as follows:

$$\text{FAR} = \frac{FN}{FN+TP}. \tag{10}$$

In our used SWaT and WADI datasets, each sample comprises all the sensor and actuator values of corresponding control systems at each second. Therefore, in the experiment for evaluating the FAR, we separately calculated the ratios of the number of normal samples alerted as attacks to that of all normal samples in the two datasets mentioned above. We then used the ratios to evaluate the FARs of our generated rules.

TABLE II
ATTACK CLASSIFICATION PERFORMANCE WITH 10-FOLD CROSS-VALIDATION ON THE SWAT DATASET

Fold	Accuracy	Precision	Recall	F1-score	Time/sec
1	0.9981	0.9994	0.9969	0.9981	1250
2	0.9940	0.9958	0.9875	0.9916	1251
3	0.9915	0.9851	0.9996	0.9923	1287
4	0.9968	0.9997	0.9930	0.9963	1250
5	0.9943	0.9947	0.9795	0.9870	1276
6	0.9630	0.9406	0.9354	0.9380	1228
7	0.9945	0.9843	0.9985	0.9913	1256
8	0.9951	0.9826	0.9973	0.9898	1256
9	0.9806	0.9984	0.9376	0.9670	1275
10	0.9981	0.9975	0.9970	0.9972	1260
Average	**0.9906**	**0.9878**	**0.9822**	**0.9850**	**1259**

TABLE III
ATTACK CLASSIFICATION PERFORMANCE WITH 10-FOLD CROSS-VALIDATION ON THE WADI DATASET

Fold	Accuracy	Precision	Recall	F1-score	Time/sec
1	0.9470	0.9158	0.9803	0.9470	1598
2	0.9515	0.9279	0.9801	0.9533	1585
3	0.9448	0.8658	0.9313	0.8974	1585
4	0.9852	0.9733	0.9943	0.9837	1585
5	0.9907	0.9812	0.9895	0.9853	1685
6	0.9733	0.9657	0.9889	0.9772	1676
7	0.9918	0.9735	0.9941	0.9837	1668
8	0.9952	0.9915	0.9956	0.9935	1673
9	0.9848	0.9663	0.9947	0.9803	1676
10	0.9852	0.9254	0.9109	0.9181	1620
Average	**0.9750**	**0.9486**	**0.9760**	**0.9621**	**1635**

D. Experiment Results and Analysis

1) Attack Classification Performance of Our Proposed Logic Learning Module: Tables II and III show the attack classification performance of our proposed logic learning module with 10-fold cross-validation on the SWaT and WADI datasets. The experimental result indicates that our proposed module achieves an excellent and stable performance of attack classification on the two datasets mentioned above. This result is from the 2-D tensors, which contain sufficient information, constructed in the tensor construction module. Additionally, our specially designed deep learning-based model promotes the logic learning module to effectively capture the abnormal industrial control logic. The performance of attack classification directly affects the effectiveness of the rules generated by our proposed LU-IDS. Therefore, the excellent performance in classification ensures that we can obtain more accurate rules in the rule generation module.

2) Results in Abnormal Control Logic Understanding and Rule Generation: Tables IV and V show the rules generated

TABLE IV
ATTACK DETECTION RULES GENERATED BY OUR PROPOSED LU-IDS WITH CONFIDENCE ON THE SWAT DATASET[1]

ID	Sensors and Actuators in Our Rules	Conf
1	AIT203, AIT202, FIT101, FIT201, MV101, LIT101, P204, P205	0.887
2	FIT301, AIT201, DPIT301, P102, P101, AIT202, P206, MV101, P205, P204	0.965
3	AIT503, AIT502, AIT504, FIT501, AIT501, P402	0.777
4	P201, MV201, P202, P204, P203, FIT201, P205, P206, AIT502, DPIT301, AIT503, AIT501	0.522
5	P201, FIT201, MV201, AIT201, AIT202, P102, AIT203, P202, P101	0.934
6	FIT301, DPIT301, LIT301, AIT401, P301, P302, MV304, MV303, MV302, AIT402	0.945
7	P206, FIT201, P205, P204, P203, P202, P201, MV201, DPIT301, FIT301, AIT203	0.943
8	P401, P402, P403, LIT401, P404, UV401, AIT501, FIT401, AIT502	0.940
9	LIT401, P401, P403, FIT401, AIT503, P404, AIT502, AIT501	0.707
10	AIT203, FIT101, LIT101, AIT202, FIT201, MV201	0.772
11	FIT101, LIT101, AIT504, AIT503, FIT501, FIT502	0.618
12	PIT503, AIT402, FIT601, AIT401, P302, P301, MV304, P603, P601, MV303, MV302, MV201	0.872
13	AIT503, AIT502, LIT401, P401, P402, P403, UV401, AIT504, P404, AIT501	0.543
14	FIT501, FIT502, AIT504, FIT503, P401, P402, LIT401, UV401, P403, P404	0.808
15	FIT501, AIT504, FIT502, AIT501, AIT502, UV401, P404	0.863
16	LIT101, FIT101, MV101, P101, P102, AIT201, AIT202, AIT203	0.983
17	FIT201, AIT203, MV201, P201, AIT202, P202, P203, P204, FIT101	0.842
18	MV201, FIT201, P202, P201, P203, P205, P204, P206, DPIT301, FIT301, AIT203	0.937
19	FIT501, AIT504, AIT503, AIT502, AIT501	0.854
20	P401, LIT401, FIT401, AIT402, P402, P403, P404, UV401, AIT401, AIT501	0.950
21	FIT101, LIT101, LIT301, FIT301, MV101, AIT402, P101	0.632
22	UV401, P404, P401, AIT501, LIT401, P403, P402, AIT502, AIT503	0.950
23	DPIT301, AIT501, FIT401, P206, UV401, FIT601, P205, P404, P603, P601, P302, AIT201, P204, P403	0.645
24	AIT203, FIT201, AIT202, AIT201, MV201, P201, P102, P101	0.873
25	FIT401, AIT501, LIT401, UV401, P404, P403, P402, P401, AIT502	0.992
26	P302, MV303, MV302, P301, MV304, LIT301, MV201, FIT301, DPIT301, AIT401	0.978
27	LIT101, FIT101, AIT203, FIT201, AIT202, MV201	0.963
28	LIT101, FIT101, MV101, P101, AIT202, AIT203, AIT201, P102	0.893
29	AIT504, AIT503, FIT501, AIT502, AIT501, FIT502	0.770
30	AIT203, LIT101, FIT101, AIT202, MV101, P101, FIT201	0.943
31	FIT502, FIT501, FIT503, AIT504, P302, FIT601, FIT504, P603, P601	0.753
32	FIT201, P201, MV201, AIT203, P202, AIT202, AIT201, P203	0.994
33	MV201, FIT201, P201, UV401, AIT501, AIT203, P404, P403, P402, P202, P401, LIT401, AIT201	0.887
34	MV201, FIT201, P201, P202, AIT201, P102, AIT203, AIT202, P203, LIT301	0.929
35	FIT201, P201, MV201, AIT201, P202, AIT203, P102, AIT202, AIT501, FIT501	0.877

[1] Legend: ID - rules id, Conf - confidence of the rules.

by our proposed LU-IDS with confidence. We can obverse that the confidence of most of the rules generated by our proposed LU-IDS exceeds 80%. This result indicates that we can obtain almost the same abnormal industrial control logic from the decision basis for classifying similar attack behavior and generate stable rules based on such result. We use the following three examples to describe the characteristics of these rules in detail.

Rule-ID 1: $319.738 < \text{AIT203} < 333.683 \wedge 0 < \text{FIT101} < 2.671 \wedge 0 < \text{FIT201} < 2.469 \wedge \text{MV101} = \text{On} \wedge 814 < \text{LIT101} < 871 \wedge 8.389 < \text{AIT202} < 8.467 \wedge \text{P204} = \text{Off} \wedge \text{P205} = \text{Off}$.

According to this rule, we can obverse that condition "0<FIT101" is equivalent to "MV101=On," and condition "814<LIT101" means that only when the level of the raw water tank exceeds the upper limit H (800 in SWaT), the action of turning on valve MV101 can achieve the attack effect, i.e., tank overflow. Intuitively, the valve MV101 controls the water flow to the raw water tank. When the tank level exceeds H, this valve should be closed. Therefore, the two states "MV101=On" and "LIT101>H" will never appear simultaneously in normal industrial procedures. This rule proves that the LU-IDS can learn and extract the effective relationship between sensors and actuators, and it can analyze and highlight the abnormal control logic to generate rules. In addition, the rules we generated can notify administrators that the tank is facing the risk of overflow, thereby helping them to address the anomalies caused by this attack promptly and prevent further damage to the raw water tank and other equipment.

Rule-ID 6: $2.208 < \text{FIT301} < 2.218 \wedge 19.444 < \text{DPIT301} < 19.806 \wedge 1200 < \text{LIT301} \wedge 154.447 < \text{AIT402} < 155.293 \wedge \text{P302} = \text{On} \wedge \text{P301} = \text{Off} \wedge \text{MV304} = \text{Off} \wedge \text{MV303} = \text{Off} \wedge \text{MV302} = \text{On} \wedge \text{P206} = \text{Off} \wedge 148.808 < \text{AIT401} < 148.808$.

In this rule, condition "1200 < LIT301" indicates that the level of Tank301 has exceeded the upper limit HH (1200 in SWaT). The conditions "P301=Off" and "P302=On" in this rule mean Pump301 is turned OFFand Pump302 is turned ON. This rule can be used to detect attack behaviors that destroy the pump by maliciously tampering with the value of sensor LIT301. This is because P302 is used to pump water out of the tank. If an attacker tampers with the value of LIT301 to keep the level of Tank 301 falsely higher than the upper limit HH, i.e., the real situation is that the water level in Tank 301 decreases rapidly, the PLC will control P302 to pump water out of the tank continuously. The pumps will eventually be destroyed as a result of idle running.

Rule-ID 9: $\text{P401} = \text{Off} \wedge \text{P403} = \text{Off} \wedge \text{P404} = \text{Off} \wedge 7.852 < \text{AIT501} < 8.215 \wedge 268.104 < \text{AIT503} < 271.020 \wedge \text{FIT401} = 0 \wedge 147.039 < \text{AIT502} < 269.110 \wedge 762.419 < \text{LIT401} < 850.589$.

This rule comprises highly complicated cross-stage abnormal control logic, i.e., it contains three AITs of stage 5. According to our analysis, setting the value of FIT401 to 0 under certain conditions causes P501 to shut down, thereby significantly reducing the amount of water flowing into the RO and ultimately resulting in the destruction of the RO reactor. Our proposed LU-IDS can successfully learn and analyze the implicit logical relationship between FIT401 in stage 4 and the several AITs in stage 5, after which it can generate a corresponding detection rule based on the three highlighted AITs. This type of complicated cross-stage control logic is almost impossible to understand without any help from the LU-IDS, which means that it is difficult for us to detect an attack with such complex logic only using manually designed rules.

3) The Performance of Intrusion Detection: Table VI shows the performance of intrusion detection with the rules generated by our proposed LU-IDS on the SWaT and WADI datasets. We tested our rules based on the attacks provided in the above two datasets. The results indicate that our rules can successfully detect 35/35 attacks on the SWaT dataset and 15/15 attacks on the WADI dataset. The excellent performance of intrusion detection is from our method of constructing the rules. Instead of manually designing rules, we constructed the rules by analyzing the knowledge learned by a deep learning-based model in attack classification tasks. Therefore, our rules contain highly effective information, which is almost impossible to acquire only through the experience and knowledge of experts in the field. Additionally, a few of the rules generated by our proposed LU-IDS result in a small number of false positives, which is 2.06% on the SWaT dataset and 4.29% on the WADI dataset. This result proves that although we generate a large number of rules, these rules will not cause an excessively high FARs.

4) Comparison Results of Intrusion Detection: Tables VII and VIII show the comparison results of the performance of our proposed LU-IDS to that of other SOTA methods in intrusion detection on the two datasets. To be fair to the experimental comparison, in the training process, we divided the SWaT and WADI datasets into training sets and validation sets in a ratio of 9:1 according to the time sequence of the data. This is because most SOTA methods divide the SWaT and WADI datasets according to the time sequence. Specifically, we separately divided

TABLE V
ATTACK DETECTION RULES GENERATED BY OUR PROPOSED LU-IDS WITH CONFIDENCE ON THE WADI DATASET[1]

ID	Sensors and Actuators in Our Rules	Conf
1	1_LT_001_PV, 1_AIT_003_PV, 1_LS_002_AL, 1_AIT_004_PV, 1_MV_001_STATUS, 1_LS_001_AL, 1_MV_002_STATUS, 2B_AIT_002_PV, 2B_AIT_001_PV, 2B_AIT_003_PV, 2A_AIT_004_PV, 1_FIT_001_PV	0.886
2	2_FIC_101_SP, 2_FQ_501_PV, 2_FIC_101_PV, 2_LS_301_AH, 2_LS_201_AH, 2_LS_201_AL, 2_LS_101_AL, 2_FQ_601_PV, 2_FQ_401_PV, 2_LS_101_AH, 2_FIC_201_CO, 2_FIC_101_CO, 2_FQ_301_PV	0.759
3	2_FIT_003_PV, 2_FIT_002_PV, 2_FQ_101_PV, 2_FIT_001_PV, 2_FQ_201_PV, 2_FQ_301_PV, 2_FQ_401_PV, 2_FIC_601_SP	0.861
4	2_FIC_201_PV, 2_FIC_201_SP, 2_FIC_201_CO, 1_P_003_STATUS, 2_DPIT_001_PV, 2_FIC_101_CO, 1_P_002_STATUS, 1_P_004_STATUS, 1_P_006_STATUS, 1_P_005_STATUS, 2_FIC_101_SP, 2_FIC_101_PV	0.844
5	2_LS_301_AH, 2_LS_201_AL, 2_LS_101_AL, 2_LS_201_AH, 2_LS_301_AL, 2_LS_101_AH, 2_LS_401_AH, 2_FQ_601_PV, 3_P_002_STATUS, 3_P_003_STATUS, 3_P_001_STATUS, 2_LS_401_AL, 3_P_004_STATUS, 2_FIC_301_CO, LEAK_DIFF_PRESSURE, 3_MV_003_STATUS, 3_MV_002_STATUS, 2_FQ_501_PV, 2_LS_501_AH, 3_MV_001_STATUS, 3_LT_001_PV	0.596
6	1_AIT_004_PV, 1_AIT_002_PV, 1_AIT_003_PV, 1_AIT_001_PV, 1_AIT_005_PV, 1_FIT_001_PV, 1_LS_001_AL, 2_FIC_201_PV, 2_FIC_201_CO, 1_LS_002_AL	0.844
7	3_AIT_003_PV, 3_AIT_002_PV, 2B_AIT_002_PV, 3_AIT_001_PV, 2B_AIT_003_PV, 2B_AIT_004_PV, 1_FIT_001_PV, 1_LT_001_PV, 1_LS_002_AL, 1_LS_001_AL, 1_AIT_005_PV	0.846
8	2A_AIT_004_PV, 2B_AIT_001_PV, 2B_AIT_002_PV, 2A_AIT_003_PV, 2_FQ_401_PV, 2_FQ_301_PV, 2_FQ_201_PV, 2_FIT_003_PV, 2_FQ_101_PV, 2B_AIT_003_PV	0.785
9	2_LS_101_AL, 2_LS_101_AH, 2_MCV_501_CO, 2_MCV_401_CO, 2_MCV_601_CO, 2_LS_201_AH, 2_FQ_601_PV, 2_MCV_301_CO, 2_LS_201_AL, 2_LS_301_AH, 2_MCV_201_CO, 2_LS_301_AL, 2_LS_401_AH, 2_MV_001_STATUS, 2_LS_401_AL	0.822
10	2_LS_101_AH, 2_FQ_601_PV, 2_LS_101_AL, 2_LS_401_AH, 2_LS_301_AL, 2_LS_201_AH, 2_LS_201_AL, 2_LS_301_AH, 2_LS_401_AL, 2_LS_501_AH, 2_FQ_501_PV, 2_MCV_601_CO, 2_FQ_401_PV, 2_MCV_501_CO, 2_LS_501_AL, 2_MV_001_STATUS	0.733
11	2_FIC_201_CO, 2_FIC_201_PV, 2_FIC_201_SP, 2_FIC_301_SP, 2_FIC_301_PV, 2_FIC_401_CO, 2_FIC_501_PV, 2_FIC_501_SP, 2_FIC_501_CO	0.643
12	2_DPIT_001_PV, 2_FIC_201_CO, 2_FIC_101_CO, 2_FIC_101_SP, 2_FIC_101_PV, 2_FIC_201_PV, 1_P_006_STATUS, 1_P_005_STATUS, 2_FIC_201_SP, 1_P_004_STATUS	0.850
13	2_FIC_501_PV, 2_FIC_501_SP, 2_FIC_601_CO, 2_FIC_501_CO, 2_FIC_401_PV, 2_FIC_601_PV, 2_FIC_601_SP, 2_FIT_001_PV, 2_FIC_401_SP	0.908

[1] Legend: ID - rules id, Conf - confidence of the rules, OV - off value (100 in WADI dataset), C - a specific value that remains constant over a period of time, LDP - LEAK-DIFF-PRESSURE, TCRF - TOTAL-CONS-REQUIRED-FLOW, PSSL - PLANT-START-STOP-LOG.

TABLE VI
PERFORMANCE OF OUR GENERATED RULES IN INTRUSION DETECTION TASKS ON THE SWAT AND WADI DATASETS

	SWaT		WADI	
Attack Types	Attacks	Detected	Attacks	Detected
SSSP	22	22	8	8
SSMP	5	5	2	2
MSSP	4	4	2	2
MSMP	3	3	3	3
DoS	1	1	-	-
Total	35	35	15	15
FAR	0.0206		0.0429	

TABLE VII
COMPARISON RESULTS WITH SOTA METHODS ON THE SWAT DATASET

Method	Accuracy	Precision	Recall	F1-score
W-PCA [19]	0.961	0.929	0.783	0.850
UAE [19]	0.951	0.970	0.715	0.823
1D-CNN [19]	0.953	0.923	0.782	0.847
AADS [18]	0.969	0.902	0.839	0.869
DAICS [4]	0.978	0.956	0.856	0.903
Our Proposed LU-IDS	**0.982**	**1.000**	**0.979**	**0.990**

TABLE VIII
COMPARISON RESULTS WITH SOTA METHODS ON THE WADI DATASET

Method	Accuracy	Precision	Recall	F1-score
W-PCA [19]	0.932	0.764	0.630	0.691
UAE [19]	0.936	0.906	0.617	0.734
1D-CNN [19]	0.929	0.687	0.753	0.719
AADS [18]	0.917	0.776	0.666	0.717
DAICS [4]	0.946	0.868	0.743	0.801
Our Proposed LU-IDS	**0.960**	**1.000**	**0.957**	**0.978**

the normal data and the data corresponding to each attack in the two datasets into 10-fold according to the time sequence. We then combined the first nine-fold (i.e., the top 90% of data on the time sequence) of the normal data and that of each attack data as the training sets. The last fold (i.e., the last 10% of data on the time sequence) of the normal data and that of each attack data are combined as the validation sets.

Compared to these methods, our proposed LU-IDS achieves the best performance on all the indicators. For example, compared to directly inputting the sensor and actuator values into the machine or deep learning-based models, such as W-PCA, 1D-CNN, and UAE, to identify the attacks, our proposed LU-IDS improves the F1-score by approximately 15% on the SWaT dataset. According to the experimental results, we believe that LU-IDS can achieve excellent performance in intrusion detection for the following two reasons.

1) The deep learning-based model in our logical learning module is competent in carrying out attack classification tasks. Our rules are constructed by extracting abnormal industrial control logic. Therefore, the excellent performance of the rules in intrusion detection benefits from the ability of LU-IDS to obtain effective abnormal control logic. The deep learning-based model in our logical learning module can capture the relationship between sensors and actuators from the measured element tensors containing more information, laying a foundation for effectively extracting abnormal industrial control logic.
2) The rules can filter the redundant information learned by the model and highlight the industrial abnormal control logic caused by the attack. Specifically, we input historical attack samples into the trained deep learning model and extract feature maps in deep convolutional layers to construct attack-specific mappings. The mappings can effectively highlight the difference in industrial control logic between the attacked state and the normal state of the ICS. Therefore, attack-specific mappings can help us extract abnormal control logic and construct intrusion detection rules.

E. Discussion on the Limitations of LU-IDS

Although the rules generated based on the abnormal control logic using our proposed LU-IDS demonstrate enhanced performance on intrusion detection, our generated rules still have the following limitations:

1) Although our rules have better performance on detection and FARs, constructing rules needs to learn historical attack behaviors. Therefore, the effectiveness of our rules is limited by the performance of the logic learning module on attack classification tasks.
2) Some of these rules contain rigorous judgment conditions for the measured elements, and as a result, a few rules have low versatility.
3) The rules generated by our proposed LU-IDS are closely related to the industrial production process. The industrial logic between different ICSs is hugely different, so it is necessary for us to train different logic learning modules

for LU-IDS in different industrial scenarios. After meeting this condition, our LU-IDS can construct appropriate intrusion detection rules for different industrial scenarios.

Therefore, in our future studies, we intend to fuse the rules generated by our proposed LU-IDS with manually designed rules in a certain way. In other words, we intend to generate highly effective and general intrusion detection rules by comprehensively considering the knowledge learned by the deep learning-based models and the knowledge of experts in the field.

VI. Conclusion

In this study, we proposed an IDS based on rule matching. Our proposed system took all the sensor and actuator values as input, and it output the interpretable rules to detect attacks against ICSs. Our proposed LU-IDS used a specially designed deep learning-based model to capture features from the 2-D measured element tensors automatically. Additionally, it could analyze the knowledge learned by a deep learning-based model during the classification process to understand the abnormal industrial control logic and generate detection rules. The experimental results showed that the rules generated by our proposed LU-IDS had high confidence, which meant that these rules were reliable. Moreover, we compared the performance of our proposed LU-IDS to that of SOTA methods in terms of intrusion detection. The results showed that our proposed LU-IDS could be used to successfully detect all types of attacks. In addition, our generated rules contained more information related to attacks, and as a result, they could help administrators to promptly address anomalies resulting from different types of attacks.

References

[1] D. Bhamare et al., "Cybersecurity for industrial control systems: A survey," *Comput. Secur.*, vol. 89, 2020, Art. no. 101677.

[2] S. Hong and J. Yu, "Ransomware attack analysis and countermeasures of defensive aspects," *J. Convergence Inf. Technol.*, vol. 8, no. 1, pp. 139–145, 2018.

[3] S. Adepu and A. Mathur, "Distributed attack detection in a water treatment plant: Method and case study," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 86–99, Jan./Feb. 2021.

[4] M. Abdelaty, R. Doriguzzi-Corin, and D. Siracusa, "DAICS: A deep learning solution for anomaly detection in industrial control systems," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 1117–1129, Apr.–Jun. 2022.

[5] K. Jang et al., "Adversarial autoencoder based feature learning for fault detection in industrial processes," *IEEE Trans. Ind. Inform.*, vol. 18, no. 2, pp. 827–834, Feb. 2022.

[6] Z. Zheng et al., "Wide and deep convolutional neural networks for electricity-theft detection to secure smart grids," *IEEE Trans. Ind. Inform.*, vol. 14, no. 4, pp. 1606–1615, Apr. 2018.

[7] D. Wu et al., "LSTM learning with bayesian and gaussian processing for anomaly detection in industrial IoT," *IEEE Trans. Ind. Inform.*, vol. 16, no. 8, pp. 5244–5253, Aug. 2020.

[8] X. Zhou et al., "Variational LSTM enhanced anomaly detection for industrial big data," *IEEE Trans. Ind. Inform.*, vol. 17, no. 5, pp. 3469–3477, May 2021.

[9] F. Kong et al., "Integrated generative model for industrial anomaly detection via bi-directional LSTM and attention mechanism," *IEEE Trans. Ind. Inform.*, to be published, doi: 10.1109/TII.2021.3078192.

[10] I. Ahmed et al., "Programmable logic controller forensics," *IEEE Secur. Privacy*, vol. 15, no. 6, pp. 18–24, Nov./Dec. 2017.

[11] W. Liang et al., "An industrial network intrusion detection algorithm based on multifeature data clustering optimization model," *IEEE Trans. Ind. Inform.*, vol. 16, no. 3, pp. 2063–2071, Mar. 2020.

[12] G. Bernieri, M. Conti, and F. Turrin, "Evaluation of machine learning algorithms for anomaly detection in industrial networks," in *Proc. IEEE Int. Symp. Meas. Netw.*, 2019, pp. 1–6.

[13] Q. Wang et al., "Recent advances in machine learning-based anomaly detection for industrial control networks," in *Proc. 1st Int. Conf. Ind. Artif. Intell.*, 2019, pp. 1–6.

[14] Q. Lin et al., "Tabor: A graphical model-based approach for anomaly detection in industrial control systems," in *Proc. Asia Conf. Comput. Commun. Secur.*, 2018, pp. 525–536.

[15] J. Inoue et al., "Anomaly detection for a water treatment system using unsupervised machine learning," in *Proc. IEEE Int. Conf. Data Mining Workshops*, 2017, pp. 1058–1065.

[16] D. Shalyga, P. Filonov, and A. Lavrentyev, "Anomaly detection for water treatment system based on neural network with automatic architecture optimization," 2018. [Online]. Available: http://arxiv.org/abs/1807.07282

[17] M. Kravchik and A. Shabtai, "Detecting cyber attacks in industrial control systems using convolutional neural networks," in *Proc. Workshop Cyber-Phys. Syst. Secur. Privacy*, 2018, pp. 72–83.

[18] M. Abdelaty, R. Doriguzzi-Corin, and D. Siracusa, "Aads: A noise-robust anomaly detection framework for industrial control systems," in *Proc. Int. Conf. Inf. Commun. Secur.*, Springer, 2019, pp. 53–70.

[19] M. Kravchik and A. Shabtai, "Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2179–2197, Jul./Aug. 2022.

[20] D. Li et al., "Mad-gan: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw.*, Springer, 2019, pp. 703–716.

[21] H. Farsi, A. Fanian, and Z. Taghiyarrenani, "A novel online state-based anomaly detection system for process control networks," *Int. J. Crit. Infrastruct. Protection*, vol. 27, 2019, Art. no. 100323.

[22] K. He et al., "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[23] G. Montavon et al., "Layer-wise relevance propagation: An overview," *Explainable AI: Interpreting, Explaining Visualizing Deep Learn.*, vol. 11700, pp. 193–209, 2019.

[24] J. Goh et al., "A dataset to support research in the design of secure water treatment systems," in *Proc. Int. Conf. Crit. Inf. Infrastructures Secur.*, Springer, 2016, pp. 88–99.

[25] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "Wadi: A water distribution testbed for research in the design of secure cyber physical systems," in *Proc. 3rd Int. Workshop Cyber- Phys. Syst. Smart Water Netw.*, 2017, pp. 25–28.

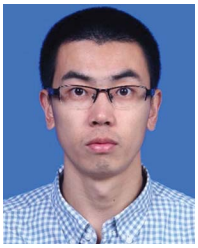
Motong Sun received the B.E. degree in information security from the Beijing University of Technology, Beijing, China, in 2019. He is currently working toward the M.S. degree in information security with the Beijing University of Technology.

He also explored in the field of deep learning and model interpretation. His research interests include intrusion detection, logical attribution, and anomaly recognition.

Yingxu Lai (Member, IEEE) received the Ph.D. degree in computer science and technology from the Chinese Academy of Sciences, Beijing, China, in 2003.

She joined the College of Computer Science, Beijing University of Technology, in 2003, and is current a Full Professor. She was a Visiting Scholar with Arizona State University, Tempe, AZ, USA, from 2013 to 2014. She has had more than 70 papers published in various international journals and conferences. Her research interest covers cloud computing, network security, edge computing, and trusted computing.

Prof. is currently an Associate-Editor of the *Journal of Artificial Intelligence and Technology*.



Yipeng Wang (Member, IEEE) received the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), Beijing, China, in 2014.

He is currently an Associate Professor with the Faculty of Information Technology, Beijing University of Technology, Beijing. He has published more than 40 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY (TIFS), IEEE/ACM TRANSACTIONS ON NETWORKING (TON), IEEE INTERNATIONAL CONFERENCE ON NETWORK PROTOCOLS, AND COMPUTER NETWORKS. His research interests are in networking, network security and machine learning.

Dr. Wang serves as a Regular Reviewer of IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE/ACM TRANSACTIONS ON NETWORKING and *Elsevier Computer Networks* etc. And he serves as a Program Committee of IJCAI-21 and IJCAI-22. He was a recipient of the Best Paper Award at IEEE International Conference on Network Protocols (ICNP) on his protocol format inference technology.

Jing Liu (Member, IEEE) received the Ph.D. degree from the Beijing University of Technology, Beijing, China, in 2017.

She is currently a Lecturer with the Faculty of Information Technology, Beijing University of Technology. She participated in the development of a number of high-level prototype systems, such as the software-defined network terminal access prototype system based on trusted computing, the industrial Internet platform security reference model, etc. She has had more than 30 papers authored or coauthored in various international journals and conferences. Her research interest covers network security, ICS security, edge computing, and trusted computing.

Dr. Liu is a reviewer for several international journals and conferences. She is a member of the China Computer Federation and a member of IEEE.

Beifeng Mao received the B.E. degree in information security from the Beijing University of Technology, Beijing, China, in 2019. He is currently working toward the M.S. degree in information security with the Beijing University of Technology.

His research interests include intrusion detection, multistep attack detection, and deep learning.

Haoran Gu received the B.E. degree in information security from the Beijing University of Technology, Beijing, China, in 2019. He is currently working toward the M.S. degree in information security with the Beijing University of Technology.

Currently, a paper has been published in *Neural Computing and Applications* (NCA). His research interests include intrusion detection and deep learning.

Mr. Gu is one of the CCF members, won the title of excellent college graduates in Beijing.